

Memoria del Proyecto: ShinyVillage

Asignatura: Proyecto Intermodular DAM

Alumno: Cristina García Quintero

Fecha: Marzo 2026

Repositorio: [URL del repositorio GitHub](#)

aquí el resumen en español y en inglés

1. Descripción del proyecto

1.1 Introducción

ShinyVillage es un videojuego de rol en dos dimensiones (RPG 2D) que fue creado con el motor Unity y cuya programación se realizó en C#. La propuesta principal es poner al jugador en la dirección de una aldea, donde tiene que administrar los recursos, cultivar su granja y examinar el ambiente desde un punto de vista cenital típico del género.

El jugador maneja a un personaje que se mueve sin restricciones por el mapa, recoge objetos del entorno, los guarda en un inventario y realiza acciones con aquellos elementos del terreno que son interactivos, como las celdas agrícolas. El personaje es seguido de manera constante por la cámara. Como el progreso se almacena en una base de datos local (SQLite), el jugador tiene la posibilidad de crear múltiples partidas independientes, reanudarlas cuando quiera o eliminarlas desde el menú.

El ciclo de juego proyectado incorpora procedimientos relacionados con la agricultura (cultivar, regar y recoger productos en tiles), manejo del inventario (recoger, amontonar y soltar artículos) e interacción con el mapa a través de un sistema de tiles clasificados según su condición.

1.2 Contexto del proyecto

Aspecto	Implementación
Ámbito y entorno	El proyecto se desarrolla en un contexto académico, sin un cliente externo, donde el propio equipo actúa como desarrollador y receptor técnico del producto. El entorno de desarrollo se basa en Unity con C# , utilizando SQLite integrado mediante NuGetForUnity para la persistencia de datos, mientras que el control de versiones se gestiona con Git mediante ramas de desarrollo. El proyecto parte desde cero con el objetivo de aprender desarrollo de videojuegos y aplicar una arquitectura sólida. Para ello se plantean como elementos clave un sistema de guardado seguro del estado de la partida, un inventario con gestión visual de espacios, movimiento del personaje con animaciones fluidas, un sistema de losetas interactivas basado en Tilemap y una estructura escalable que permita añadir nuevas mecánicas sin modificar el núcleo del sistema.

Aspecto	Implementación
Solución y justificación	La solución elegida estructura el juego en sistemas independientes conectados mediante patrones de diseño. Se utiliza el patrón Singleton en gestores como GameManager , DatabaseManager y SaveGameManager para garantizar una única instancia global en Unity . La persistencia se gestiona con SQLite , lo que permite organizar los datos de forma relacional y facilitar consultas y ampliaciones del sistema. Además, el patrón Repository separa el acceso a la base de datos de la lógica del juego. Por otro lado, el inventario se implementa con una clase independiente del motor y una interfaz de usuario separada para la visualización. Finalmente, el sistema de tiles utiliza el componente Tilemap , gestionado por un TileManager . Todo ello sigue una arquitectura en capas que separa presentación, lógica y datos para mejorar la mantenibilidad y escalabilidad del proyecto.
Destinatarios	El juego contempla como único tipo de usuario el jugador. Su rol dentro del sistema le permite crear y modificar partidas desde el menú, así como el movimiento de un personaje y sus decisiones sobre la interacción con el mapa. También puede gestionar su inventario decidiendo qué objetos soltar y cuáles soltar. No se contempla ningún rol de administrador ni perfil técnico dentro de la experiencia de juego. Toda la gestión interna (base de datos, repositorios, gestores) es transparente para el jugador y opera de forma automática en segundo plano.

1.3 Objetivo del proyecto

ShinyVillage tiene la finalidad de ofrecer al jugador una experiencia de ocio digital tranquilo, de estilo casual basado en una simple gestión de recursos y un entorno de fantasía.

La aplicación no tiene una vocación comercial en este momento porque está enmarcada en un contexto de aprendizaje y desarrollo en técnicas de programación de videojuegos.

1.4 Marco legal

Al tratarse de un videojuego de entretenimiento en fase de desarrollo académico, sin distribución comercial ni recogida de datos personales identificativos, el marco legal aplicable es reducido.

Protección de datos (RGPD / LOPDGDD)

La aplicación almacena únicamente datos de juego —nombre de personaje, progreso, posición— de forma local en el dispositivo del usuario, sin transmisión a servidores externos. El nombre del personaje es un alias libremente elegido, no vinculado a ninguna identidad real, por lo que en su estado actual no se tratan datos personales en el sentido del RGPD (Reglamento UE 2016/679) ni de la LOPDGDD (LO 3/2018). Si en el futuro se incorporasen cuentas de usuario o cualquier dato identificativo, la aplicación quedaría sujeta a dichas normas.

Propiedad intelectual

El proyecto ShinyVillage se distribuye bajo una **licencia personalizada basada en Apache License 2.0**, a la que se añade una cláusula de restricción de uso comercial. Esto significa que cualquier persona puede

usar, estudiar, modificar y redistribuir el software y sus versiones derivadas, pero **no puede hacerlo con fines comerciales** sin autorización expresa.

Esta licencia no está aprobada por la OSI (*Open Source Initiative*) al incluir dicha restricción, pero es plenamente válida desde el punto de vista legal.

Si en algún momento se desea autorizar un uso comercial concreto a un tercero, bastaría con un acuerdo escrito adicional entre las partes, sin necesidad de cambiar esta licencia.

Clasificación por edades

En caso de distribución pública, el sistema PEGI clasificaría previsiblemente el juego como **PEGI 3**, dado su contenido de fantasía sin elementos violentos ni adultos.

2. Acuerdo del proyecto

2.1 Requisitos funcionales y no funcionales

Requisitos funcionales

Capacidades que el programa debe poder ofrecer al usuario final, en este caso el jugador.

ID	Tipo	Requisito
RF-01	Funcional	El jugador puede crear una nueva partida introduciendo un nombre de personaje y un nombre de slot
RF-02	Funcional	El jugador puede cargar una partida guardada previamente desde el menú principal
RF-03	Funcional	El jugador puede borrar una partida existente, con confirmación previa
RF-04	Funcional	El jugador puede sobrescribir una partida con el estado actual de la sesión en curso
RF-05	Funcional	El personaje se desplaza por el mapa en las cuatro direcciones con animación asociada
RF-06	Funcional	El jugador puede recoger objetos del mundo e incorporarlos al inventario
RF-07	Funcional	El jugador puede consultar su inventario y eliminar objetos, que reaparecen en el mapa
RF-08	Funcional	El jugador puede interactuar con tiles del mapa pulsando la tecla de acción
RF-09	Funcional	El estado de la granja (tiles, cultivos, fases de crecimiento) se guarda y recupera por partida

Requisitos no funcionales

Este tipo de requisitos definen las condiciones y estándares de rendimiento y las restricciones del sistema

ID	Tipo	Requisito
RNF-01	No funcional	Los datos de partida se almacenan localmente en el dispositivo del usuario mediante SQLite
RNF-02	No funcional	El sistema de guardado persiste entre escenas sin pérdida de datos
RNF-03	No funcional	La arquitectura debe permitir añadir nuevos sistemas (cultivos, NPCs) sin modificar el núcleo
RNF-04	No funcional	El tiempo de carga de una partida guardada no debe ser perceptible para el usuario
RNF-05	No funcional	El juego debe ejecutarse en PC con un rendimiento estable bajo Unity

2.2 Planificación de tareas y temporalización

El desarrollo se organiza en fases en las que se incorpora un bloque completo y verificable antes de pasar a la siguiente.

Fase 1 · Fundamentos del proyecto

- └─ Configuración del entorno Unity y estructura de carpetas
- └─ Sistema de movimiento del personaje y seguimiento de cámara
- └─ Implementación del Tilemap e interacciones básicas

Fase 2 · Sistema de inventario

- └─ Clase Inventory y lógica de slots (añadir, apilar, eliminar)
- └─ UI del inventario (Inventory_UI, Slot_UI)
- └─ Mecánica de soltar objetos al mundo

Fase 3 · Persistencia y base de datos

- └─ Integración de SQLite mediante NuGetForUnity
- └─ DatabaseManager: conexión, tablas y operaciones CRUD
- └─ Repositorios: SaveSlotRepository, PlayerRepository
- └─ SaveGameManager como punto de entrada único al guardado

Fase 4 · Menú principal y gestión de partidas (En desarrollo)

- └─ MainMenuManager: nueva partida, carga y borrado
- └─ SaveSlotUI: prefab de fila con botones dinámicos
- └─ Integración completa del flujo menú → juego → guardado

Fase 5 · Sistema de granja (en desarrollo)

- └─ FarmRepository y FarmTileRepository
- └─ Lógica de cultivo: plantar, regar, cosechar
- └─ Persistencia del estado de cada tile por partida

2.3 Metodología a seguir

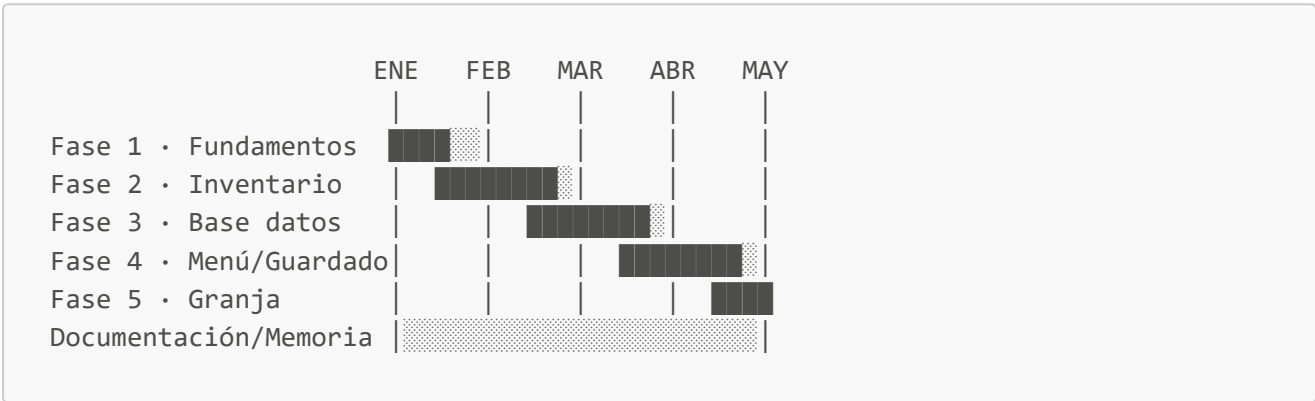
El proyecto adopta una metodología **iterativa e incremental** inspirada en Scrum, que se adapta a un equipo de desarrollo pequeño y a un contexto académico.

Cada fase de la planificación equivale a un sprint con un entregable funcional al final (tareas).

El desarrollo se gestiona mediante Git con ramas independientes por funcionalidad, integrando los cambios a la rama principal una vez verificados. Esta aproximación permite detectar errores de integración de forma temprana y mantener en todo momento una versión jugable del proyecto.

2.4 Temporalización

El diagrama de Gantt siguiente refleja la distribución temporal estimada del proyecto a lo largo del ciclo de desarrollo:



Cada fase tiene una duración aproximada de dos a tres semanas, con solapamiento en la etapa de documentación, que se mantiene activa durante todo el desarrollo.

2.5 Presupuesto

El proyecto no tiene coste económico directo, al desarrollarse en un contexto formativo con herramientas de acceso libre o gratuito para uso académico.

Concepto	Coste
Unity uso personal	0 €
SQLite + NuGetForUnity	0 €
Assets gráficos (libres de uso creados por CupNooble)	0 €
Control de versiones (Git y GitHub)	0 €
Total	0 €

El coste real del proyecto se mide en horas de desarrollo. Estimando una dedicación media de 10 horas semanales durante 5 meses, el esfuerzo total asciende a aproximadamente **200 horas**.

2.6 Contrato y pliego de condiciones/licencia

El proyecto se distribuye con una licencia **Apache 2.0 con una cláusula no comercial** que se describe en el [apartado correspondiente de esta memoria](#).

Como se trata de de un proyecto sin un cliente externo, no hay contrato de prestación de servicios. El acuerdo de desarrollo queda recogido en los terminos de entregas del centro formativo.

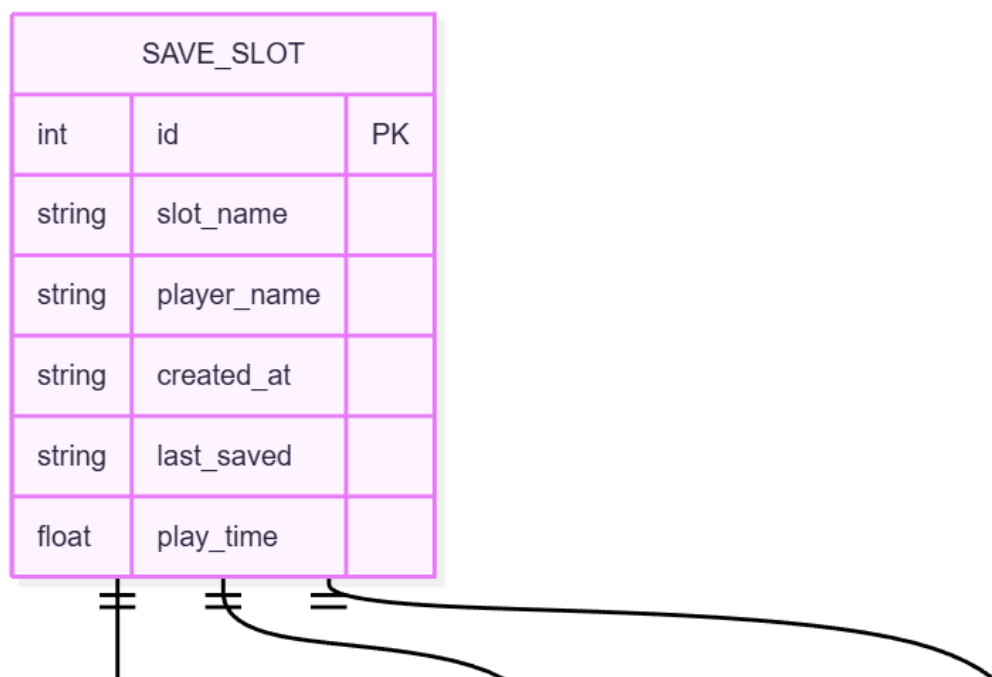
2.7 Análisis de riesgos

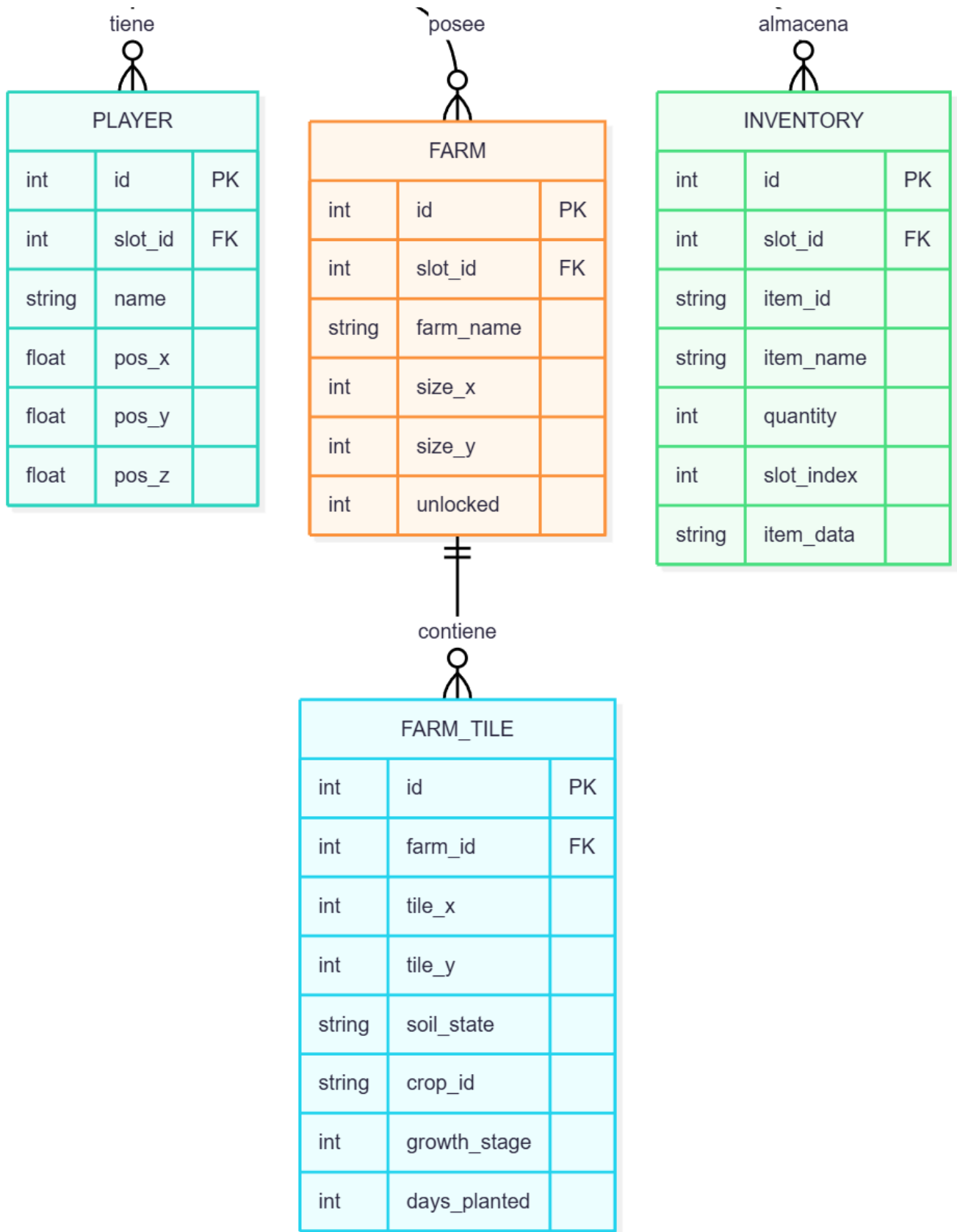
Riesgo	Probabilidad	Impacto	Mitigación
Corrupción de la base de datos SQLite	Baja	Alto	Cierre controlado de conexión en <code>OnApplicationQuit</code> y <code>OnDestroy</code>
Incompatibilidad de la librería SQLite con la versión de Unity	Media	Alto	Uso de NuGetForUnity con versión fijada y probada
Deuda técnica por crecimiento no planificado del código	Media	Medio	Arquitectura en capas con patrón Repository desde el inicio
Pérdida de progreso en el repositorio Git	Baja	Alto	Ramas por funcionalidad y commits frecuentes
Falta de tiempo para completar el sistema de granja	Media	Medio	El núcleo del juego es funcional sin él; se plantea como fase ampliable

3. Análisis y diseño

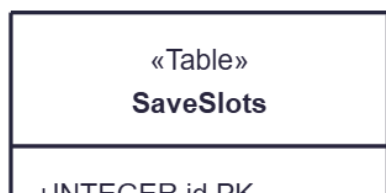
3.1 Modelado de datos

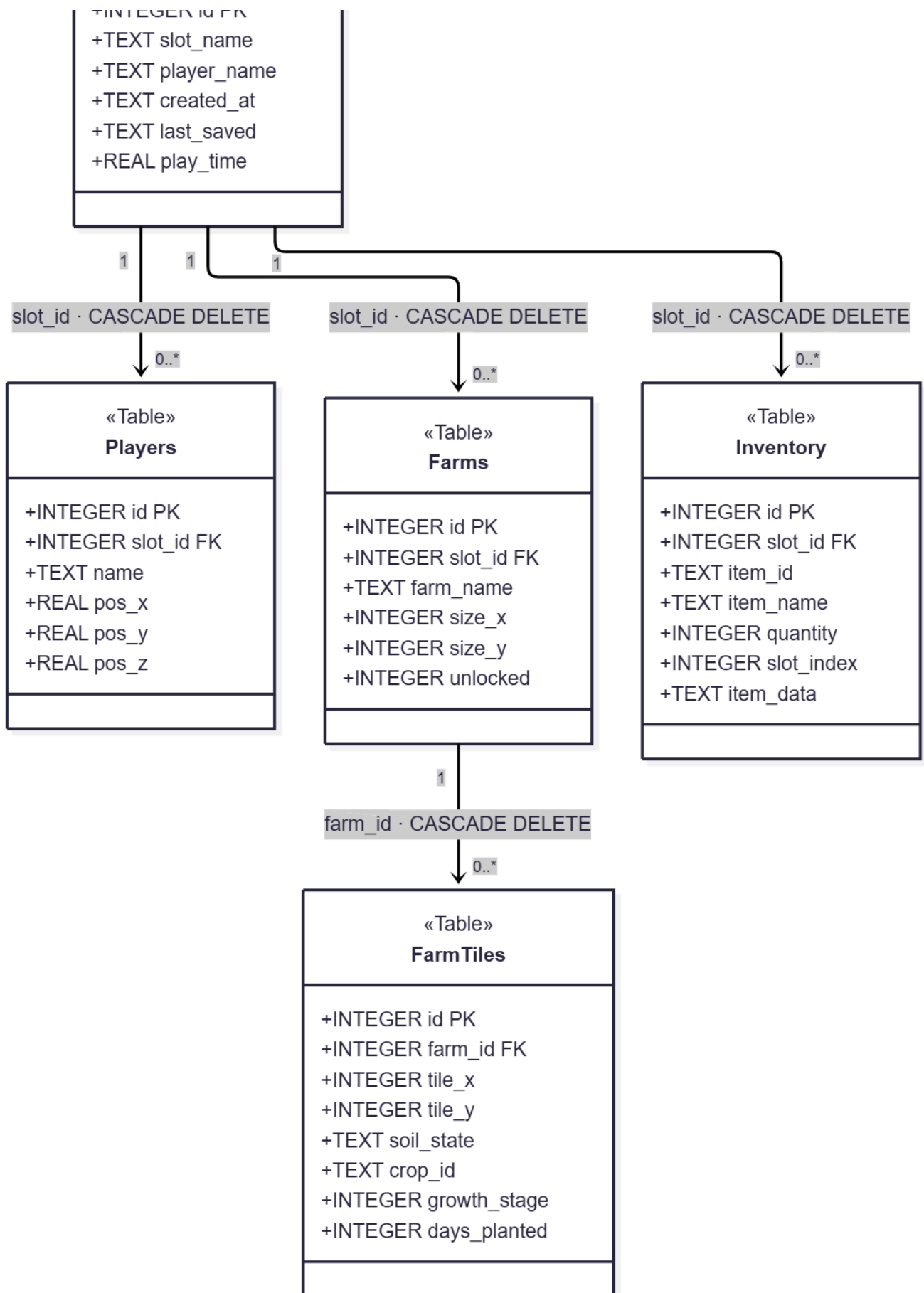
3.1.1 Modelo E/R





3.1.2 Modelo relacional del sistema de guardado





3.1.3 Script de la creación de la BBDD (SQLite)


```

-- =====
-- SCRIPT DE CREACIÓN DE TABLAS – savegame.db
-- =====

-- Tabla raíz: cada fila representa una partida guardada
CREATE TABLE IF NOT EXISTS SaveSlots (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    slot_name     TEXT      NOT NULL,
    player_name   TEXT      NOT NULL,
    created_at    TEXT      NOT NULL,
    last_saved    TEXT      NOT NULL,
    play_time     REAL      DEFAULT 0
);

-- Datos de posición del jugador, ligados a un slot
CREATE TABLE IF NOT EXISTS Players (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    slot_id       INTEGER NOT NULL REFERENCES SaveSlots(id) ON DELETE CASCADE,
    name          TEXT      NOT NULL,
    pos_x         REAL      DEFAULT 0,
    pos_y         REAL      DEFAULT 0,
    pos_z         REAL      DEFAULT 0
);

-- Granjas que pertenecen a un slot
CREATE TABLE IF NOT EXISTS Farms (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    slot_id       INTEGER NOT NULL REFERENCES SaveSlots(id) ON DELETE CASCADE,
    farm_name     TEXT      NOT NULL,
    size_x        INTEGER DEFAULT 10,
    size_y        INTEGER DEFAULT 10,
    unlocked      INTEGER DEFAULT 1  -- 1 = desbloqueada, 0 = bloqueada
);

-- Estado de cada celda/tile dentro de una granja
CREATE TABLE IF NOT EXISTS FarmTiles (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    farm_id       INTEGER NOT NULL REFERENCES Farms(id) ON DELETE CASCADE,
    tile_x        INTEGER NOT NULL,
    tile_y        INTEGER NOT NULL,
    soil_state    TEXT      DEFAULT 'dry',  -- dry | watered | fertilized
    crop_id       TEXT      DEFAULT '',
    growth_stage  INTEGER DEFAULT 0,
    days_planted  INTEGER DEFAULT 0
);

-- Inventario del jugador (un ítem por fila)
CREATE TABLE IF NOT EXISTS Inventory (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    slot_id       INTEGER NOT NULL REFERENCES SaveSlots(id) ON DELETE CASCADE,
    item_id       TEXT      NOT NULL,
    item_name     TEXT      NOT NULL,

```

```

quantity    INTEGER DEFAULT 1,
slot_index  INTEGER DEFAULT -1,
item_data   TEXT    DEFAULT '{}' -- JSON para datos extra
);

```

3.2 Análisis y diseño funcional

3.2.1 UML (Clases, secuencia y casos de uso)

Diagramas de clases

Diagrama general

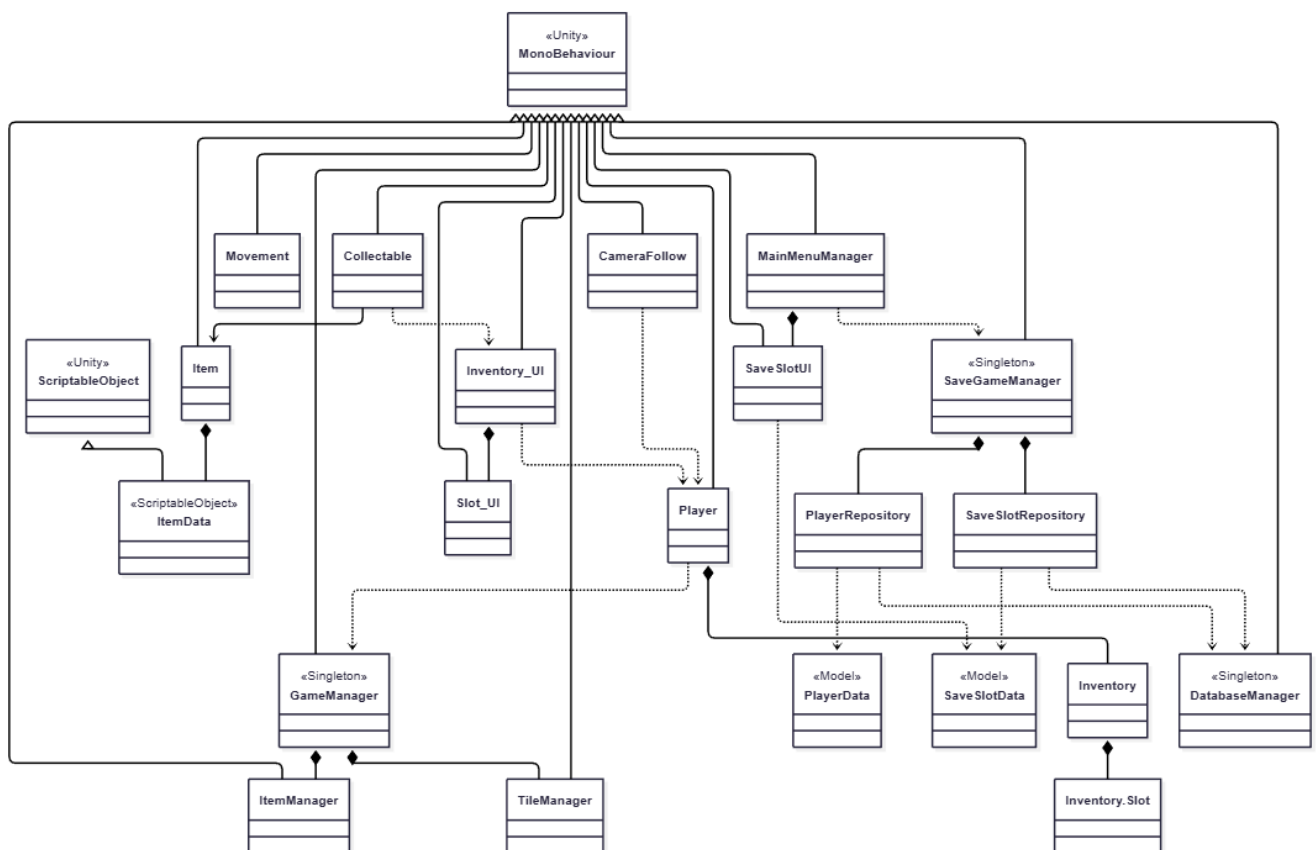


Diagrama desglosado: Gameplay

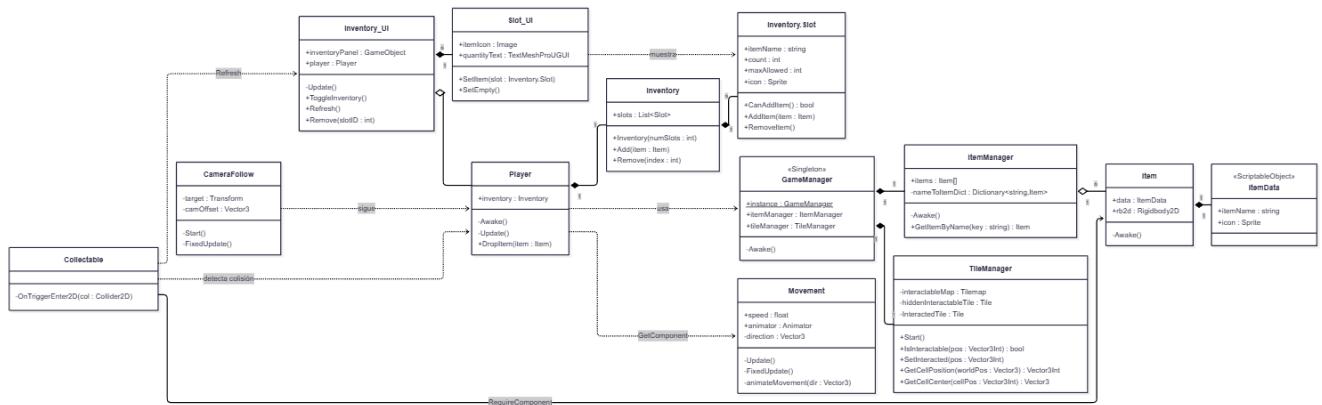
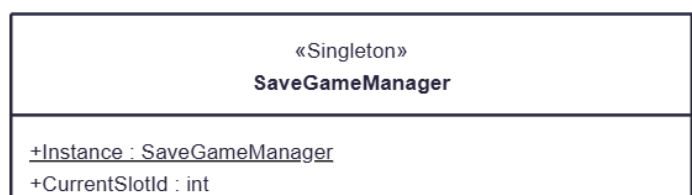


Diagrama desglosado: Saving System (Guardado)



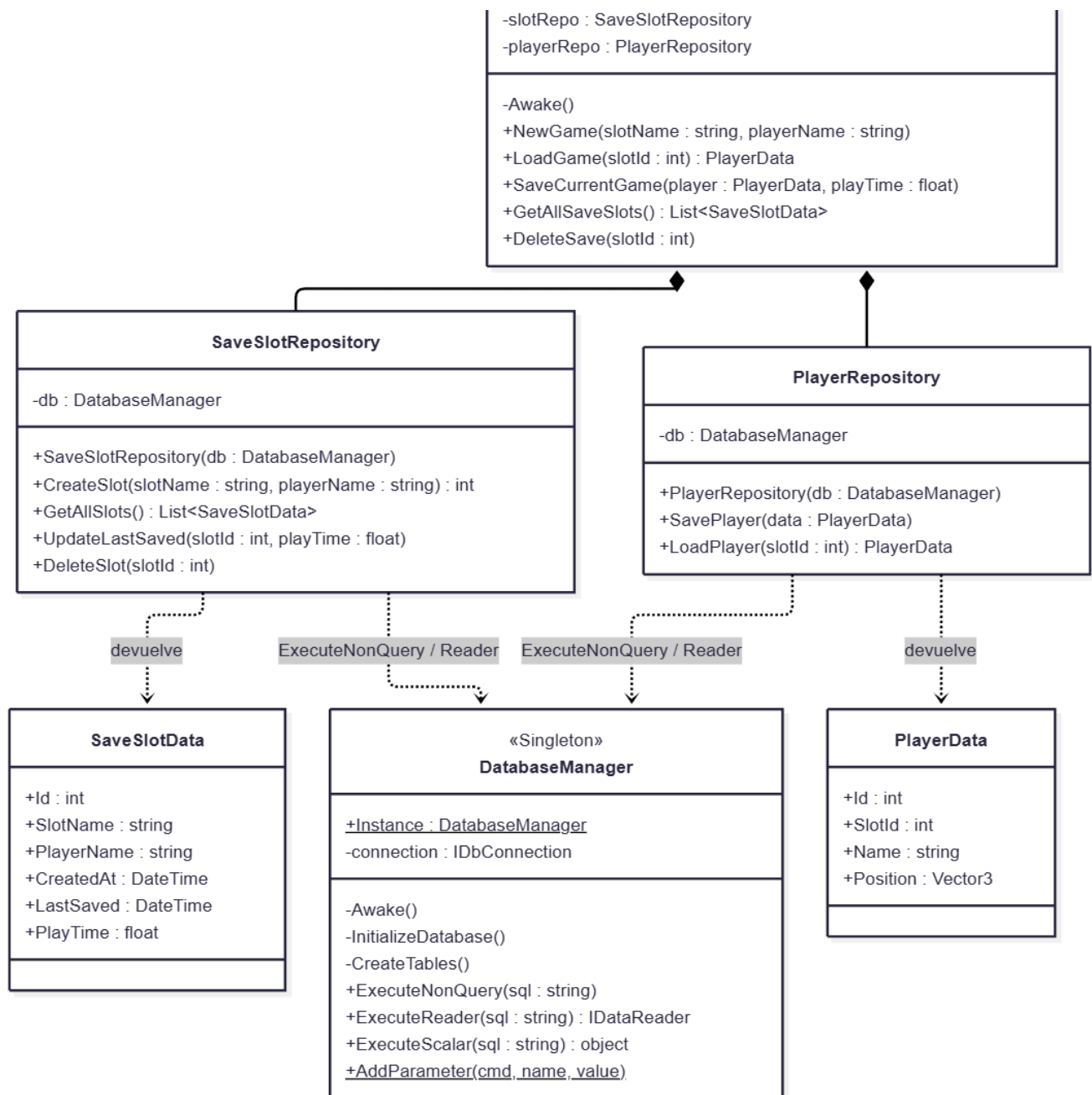
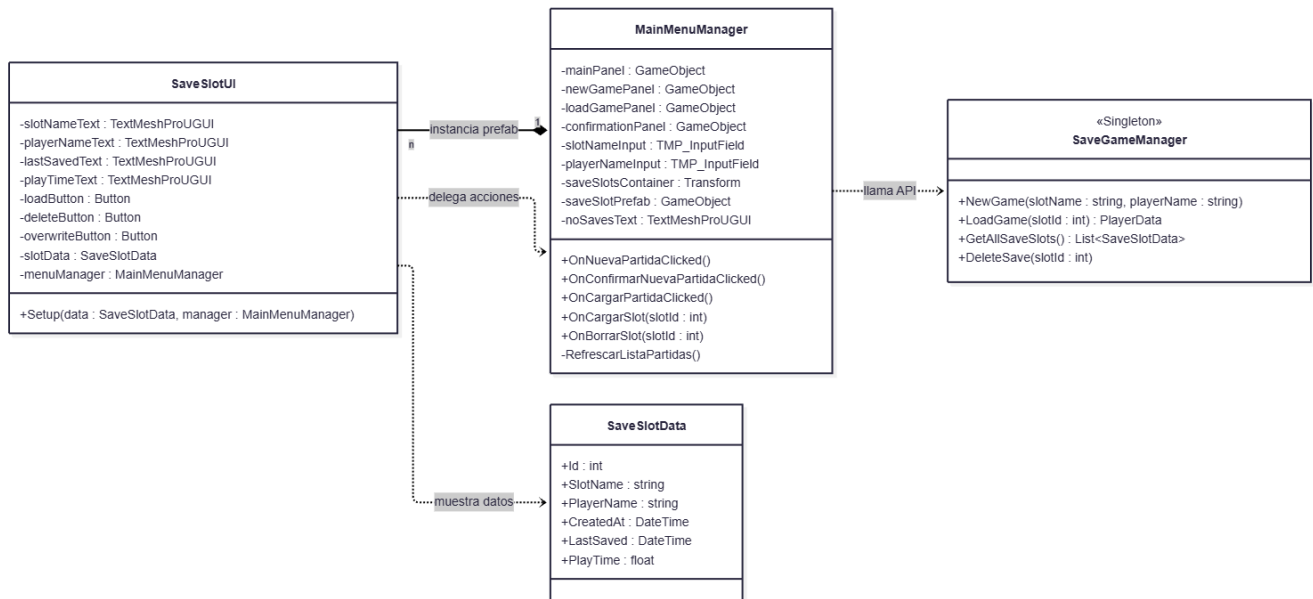
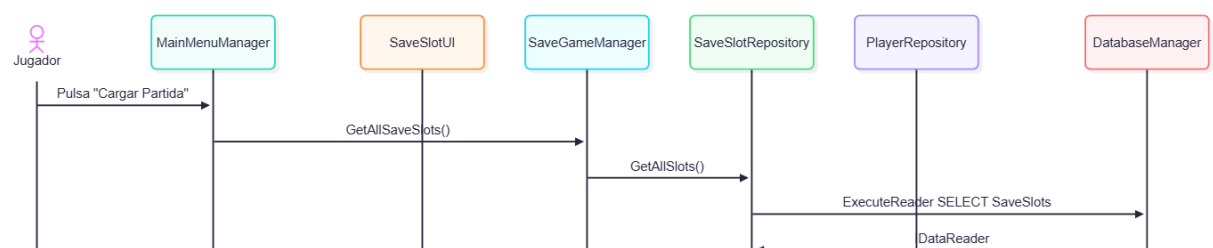
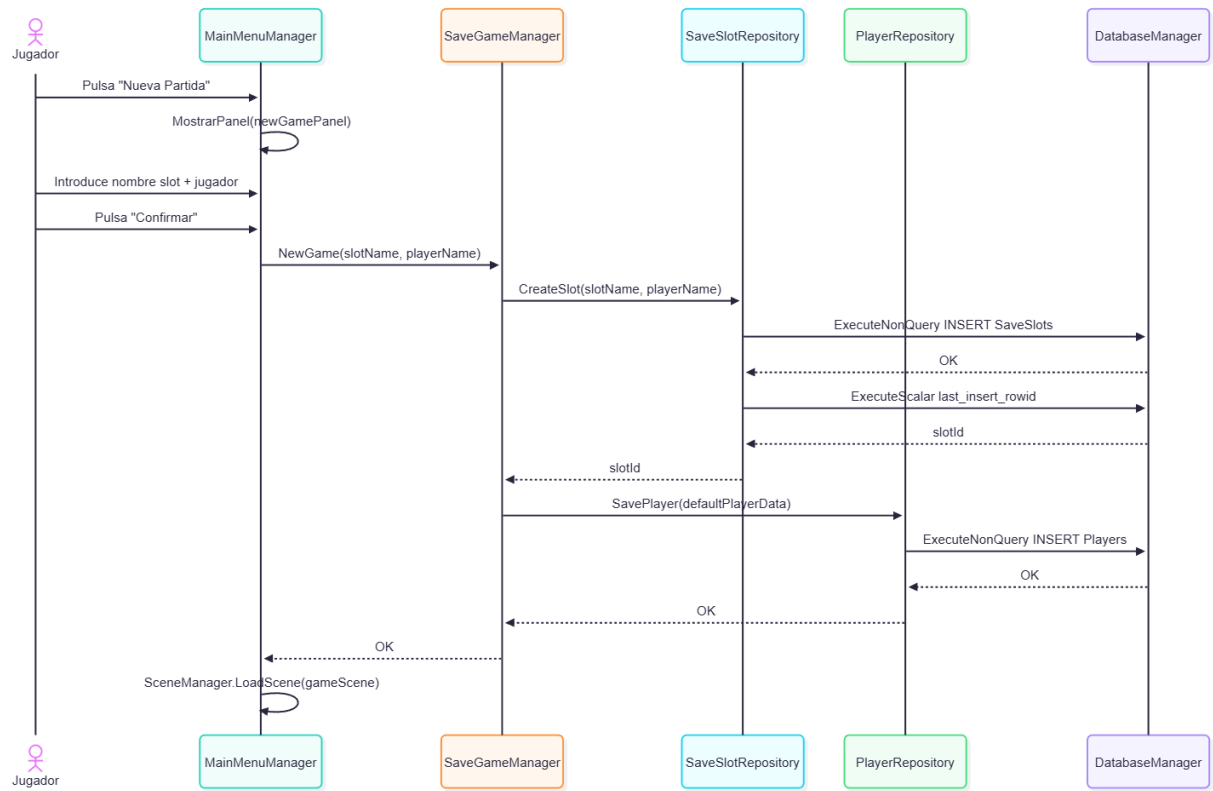


Diagrama desglosado: Main Menu



Diagramas de secuencia



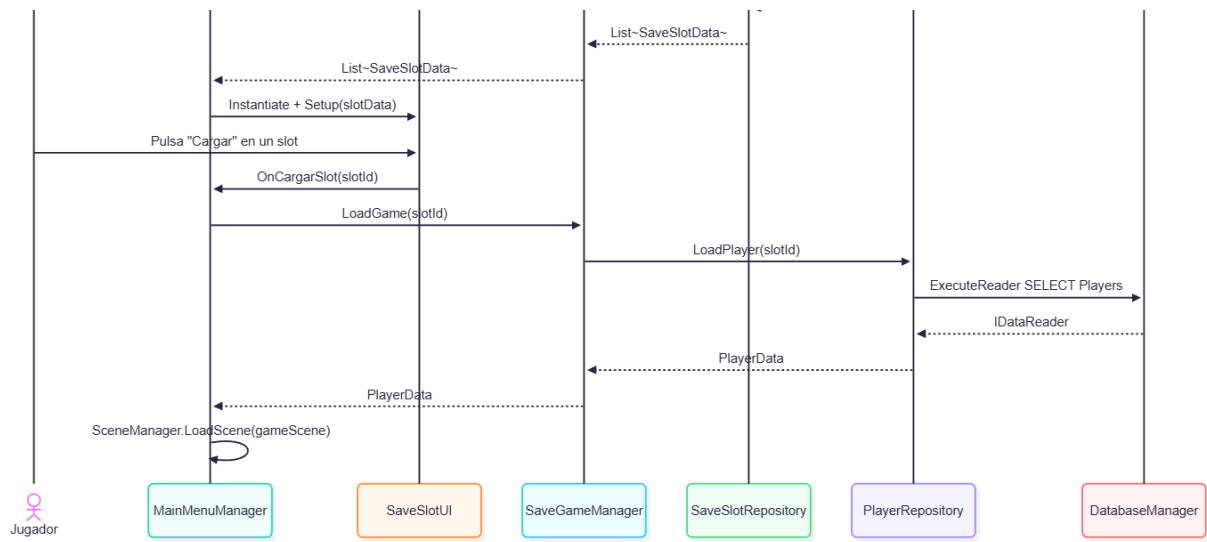
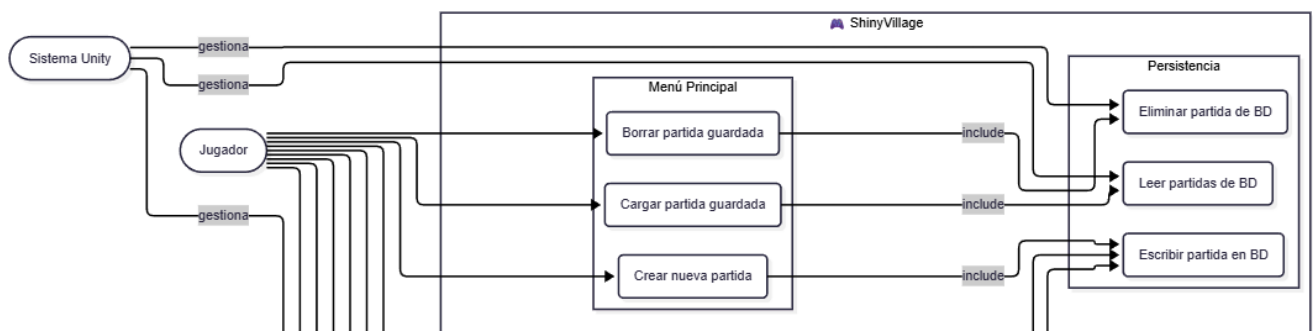
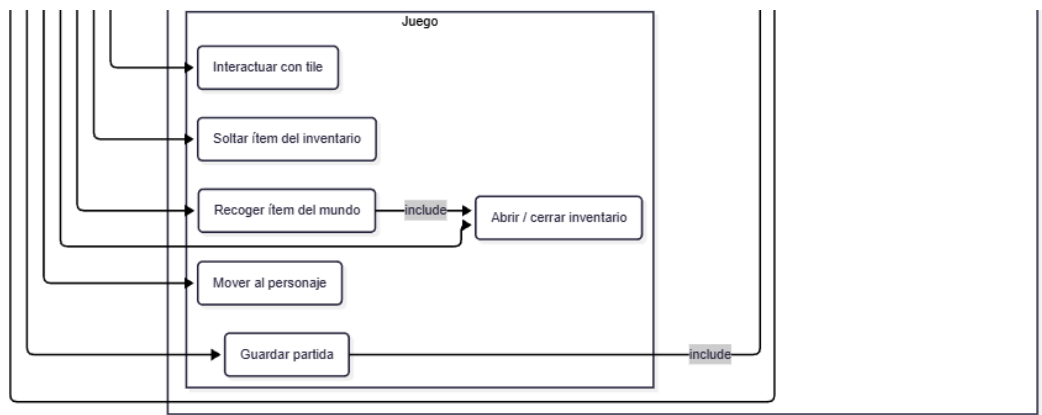
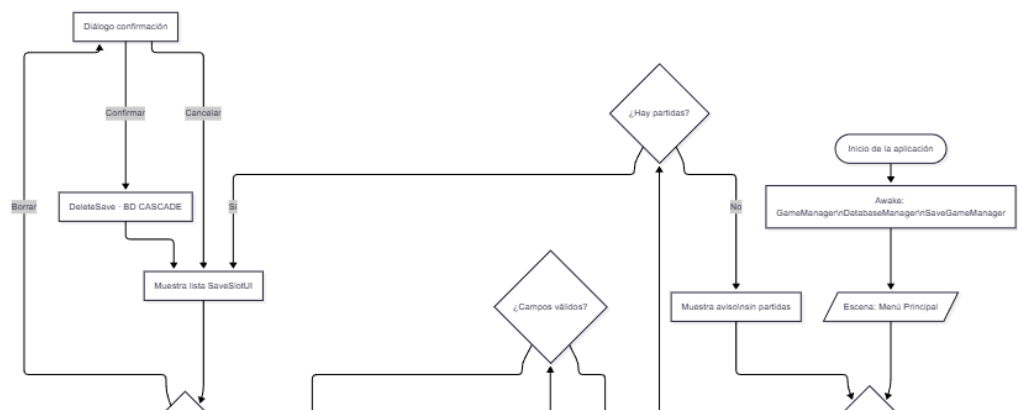


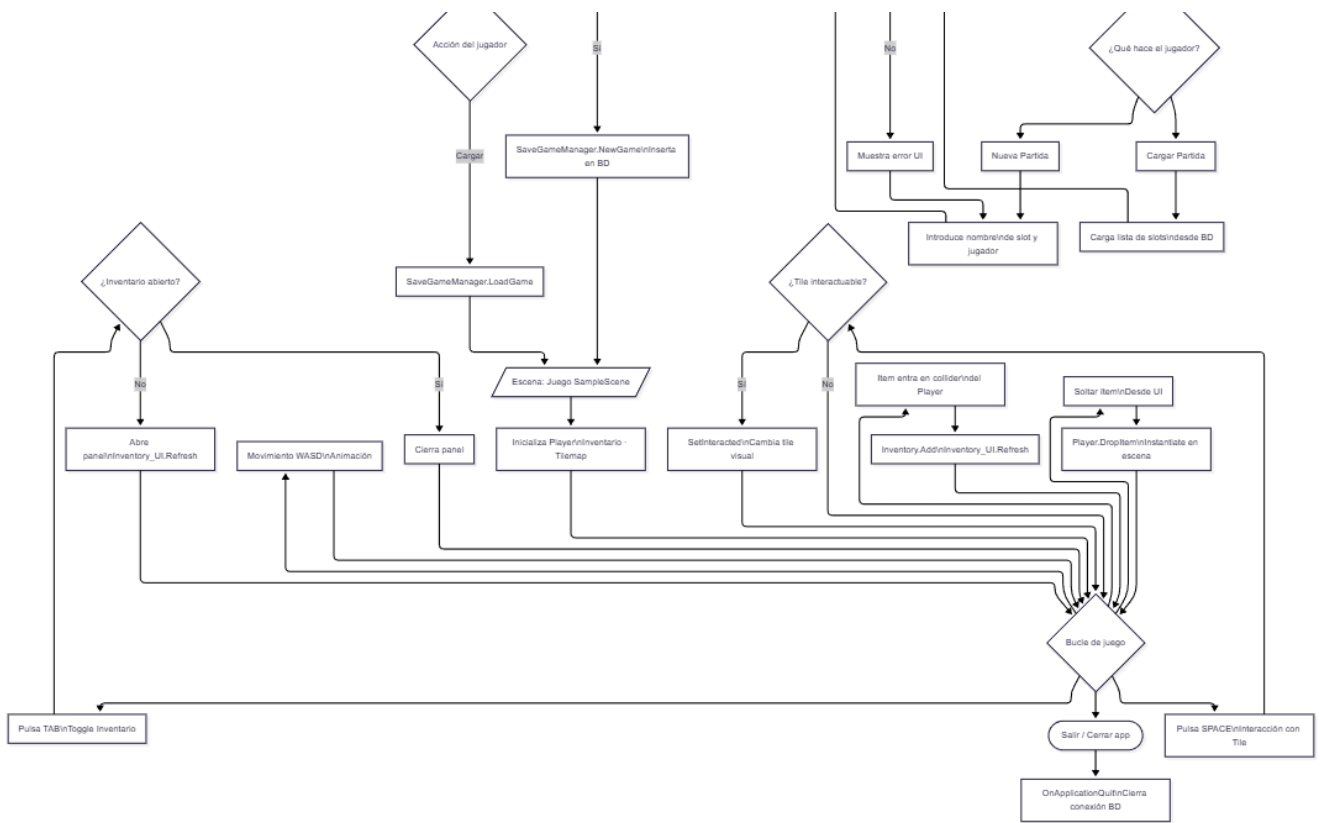
Diagrama de casos de uso



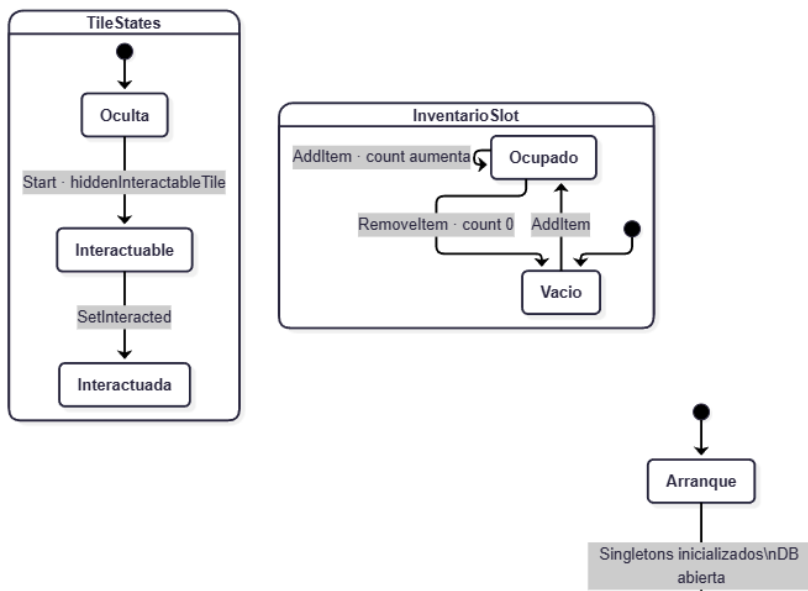


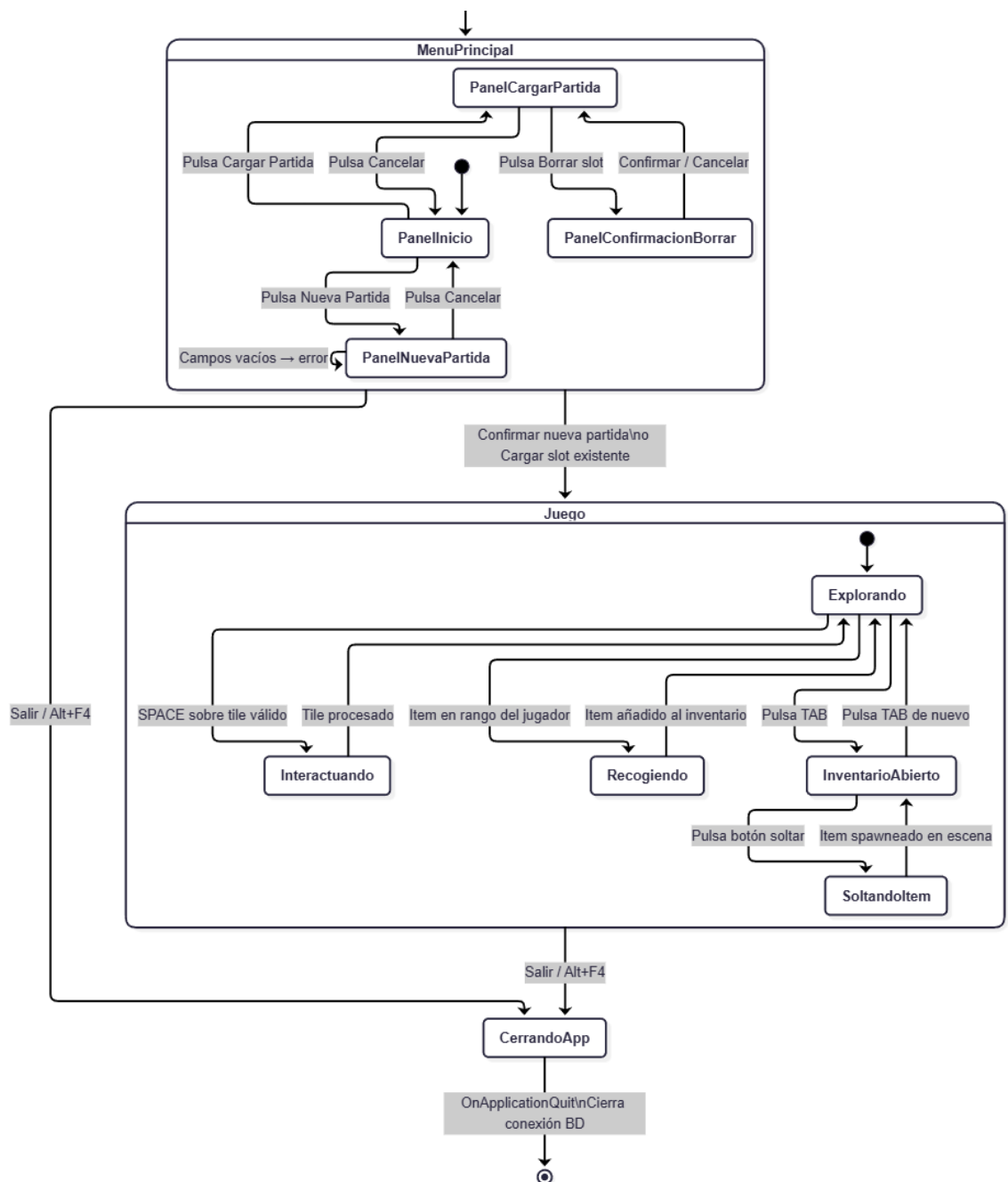
3.2.2 Diagrama de flujo



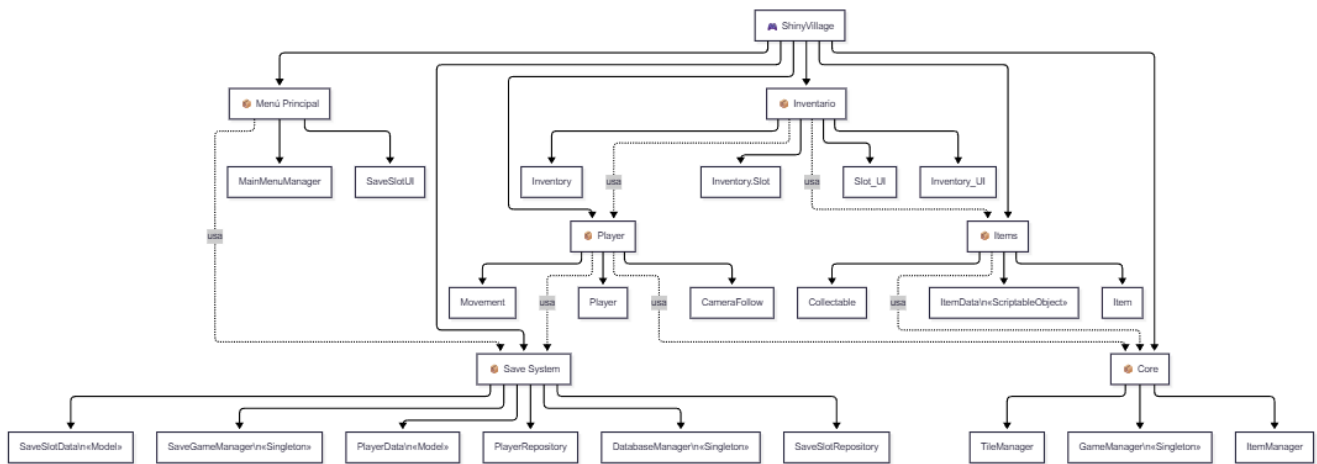


3.2.3 Diagrama de estados





3.2.4 Descomposición en módulos



3.3 Análisis y diseño de la interfaz

3.3.1 Mockups

3.3.2 Diseño visual

3.4 Análisis y diseño de la arquitectura

3.4.1 Tecnologías usadas y herramientas

Para el proyecto, se han tomado una serie de decisiones orientadas a la productividad y mantenibilidad del código, y también la calidad del producto final.

Tabla resumen de las herramientas y tecnologías usadas

Herramienta	Rol
Unity	Motor de juego principal
C#	Lenguaje para los scripts
Visual Studio Code	IDE
Git/Github	Control de versiones y ramas de trabajo
Mermaid/Markdown	Documentación técnica del repositorio y diagramas
SQLite	Base de datos

Motor de Juego

Unity se ha seleccionado como motor de juego por ser un entorno completo para desarrollar RPGs 2D. Posee sistemas integrados para el control de Tilemap, física 2D y UI Canvas, por lo que no se necesitan implementaciones externas.

También la cantidad de documentación y la comunidad son extremadamente extensas, lo que reduce el tiempo en la resolución de problemas.

La alternativa inicial fue Godot 4 aunque el sistema de assets y la lógica 2D no están tan integradas y resultó complicado aprender a manejarlas al inicio.

Lenguaje: C# / .NET Standard

C# es el único lenguaje de scripting que soporta Unity. Se ha decidido esta opción frente al Visual Scripting (Instalado en el entorno) porque puede ser versionado con git de forma muy limpia, además de que permite utilizar patrones como Singleton o Repository de manera explícita.

De igual manera, C# es uno de los lenguajes estándar en la industria del videojuego junto a C++.

Entorno de desarrollo

Se ha elegido VS Code frente a JetBrains Rider principalmente porque es gratuito y tiene una integración fluida con Git y Github. Además posee extensiones específicas para las necesidades de este proyecto como GitLens, MarkdownAllInOne y la extensión oficial de Unity.

Debido a la naturaleza del proyecto como proyecto formativo y académico Git y GitHub son herramientas estándar y se adaptan perfectamente con el IDE.

La trazabilidad de cada cambio, las ramas de trabajo y el historial de commits y decisiones lo hacen una de las herramientas más importantes en el proyecto.

La estrategia que se ha usado en las ramas es la siguiente:

- main: Rama estable que recibe merges de las funcionalidades completadas y probadas
- feature-X: Rama por funcionalidad en desarrollo

Tecnologías para la documentación

Toda la documentación técnica del proyecto se escribe en Markdown por su formato de texto plano, versionable con Git y por su renderizado nativo en GitHub, de manera que la propia memoria del proyecto se expone en la [Wiki](#) del mismo mediante un [Workflow](#) que tras cada commit que modifique ese archivo concreto, realiza una actualización de la wiki de forma automatizada.

Paquetes y dependencias

NuGetForUnity

NuGetForUnity es un gestor de paquetes NuGet integrado en el Editor de Unity. Se usa para instalar System.Data.SQLite, la librería de acceso a bases de datos SQLite para .NET.

La alternativa habitual (descargar las DLLs manualmente y añadirlas a Assets/Plugins) es propensa a errores de versión y difícil de mantener.

Aquí está la versión redactada en prosa, en tercera persona:

SQLite

SQLite se emplea como motor de base de datos para el sistema de partidas guardadas. Al tratarse de una solución embebida, no requiere servidor externo ni configuración de red: la base de datos es un único archivo `.db` almacenado en el dispositivo del jugador. Esto la convierte en una opción mucho más robusta que `PlayerPrefs`, que solo admite tipos primitivos y no sobrevive a reinstalaciones. Al ser SQL estándar, permite ejecutar consultas complejas para cargar partidas, listar slots y gestionar el inventario de forma eficiente, y lo hace de manera multiplataforma sin cambios en el código, tanto en Windows como en macOS y Linux.

La arquitectura de acceso sigue el patrón Repository (`SaveSlotRepository`, `PlayerRepository`), que separa la lógica de negocio del acceso a datos. Esta decisión facilita sustituir el motor de base de datos en el futuro sin necesidad de modificar el resto del código.

New Input System

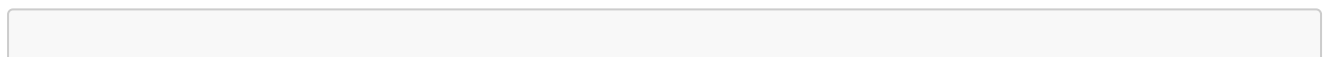
El nuevo Input System de Unity se emplea en lugar del sistema legacy (`Input.GetKey`) por ser el único que recibirá mantenimiento a largo plazo. A diferencia del sistema anterior, permite mapear controles de forma flexible desde un asset centralizado (`InputActions`), de modo que teclado y ratón se gestionan desde un único punto de configuración.

3.4.2 Arquitectura de los componentes

Estructura física: carpetas del proyecto

Con todo el código real del proyecto analizado, aquí está la explicación completa de la estructura lógica y física:

La estructura de carpetas sigue las convenciones estándar de Unity, donde todo el código fuente reside bajo `Assets/Scripts/` organizado por responsabilidad:



```

Assets/
├── Scripts/
│   ├── Database/
│   │   ├── DatabaseManager.cs
│   │   ├── SaveGameManager.cs
│   │   ├── SaveSlotRepository.cs
│   │   └── PlayerRepository.cs
│   ├── UI/
│   │   ├── Inventory_UI.cs
│   │   └── Slot_UI.cs
│   ├── ScriptableObject/
│   │   └── ItemData.cs
│   ├── Player.cs
│   ├── Movement.cs
│   ├── CameraFollow.cs
│   ├── Item.cs
│   ├── Collectable.cs
│   ├── Inventory.cs
│   ├── ItemManager.cs
│   ├── TileManager.cs
│   └── GameManager.cs
├── Scenes/
│   └── SampleScene.unity
├── Packages/          ← DLLs de NuGet (SQLite)
└── InputSystem_Actions.inputactions

```

La separación en subcarpetas refleja directamente la separación de responsabilidades: **Database/** agrupa todo lo relacionado con persistencia, **UI/** agrupa las vistas, y **ScriptableObject/** contiene los activos de datos. Los scripts que no encajan en una subcarpeta específica —**Player**, **Movement**, **GameManager**— son componentes de juego de uso general y permanecen en la raíz de **Scripts/**.

Estructura lógica: capas y patrones

El proyecto no implementa MVC de forma estricta, pero sí una arquitectura en capas equivalente:

- **Capa de datos (Modelo)**

La capa más baja gestiona los datos puros, sin lógica de juego ni presentación. Se compone de tres elementos diferenciados:

ItemData (ScriptableObject) es el modelo de un ítem en su forma más pura: solo contiene **itemName** y **icon**. Al ser un **ScriptableObject**, existe como activo en el proyecto y puede asignarse en el Inspector, desacoplando completamente la definición del ítem de cualquier lógica de juego.

Inventory y **Inventory.Slot** son clases de datos puras (no heredan de **MonoBehaviour**) que representan el estado del inventario. **Slot** encapsula el nombre del ítem, su cantidad, el máximo permitido y el icono. Al estar marcadas con **[System.Serializable]**, Unity puede mostrarlas en el Inspector para facilitar la depuración.

Los modelos de base de datos (*SaveSlotData*, *PlayerData*) son clases serializables planas que representan filas de la base de datos. No contienen lógica: solo transportan datos entre la capa de persistencia y el resto del sistema.

- **Capa de persistencia (Repository)**

Entre el modelo y la lógica de juego existe una capa intermedia dedicada exclusivamente al acceso a datos persistentes. Esta capa implementa el **patrón Repository**:

DatabaseManager actúa como la infraestructura de conexión: abre la base de datos SQLite, crea las tablas si no existen y expone tres métodos genéricos (*ExecuteNonQuery*, *ExecuteReader*, *ExecuteScalar*) que el resto de repositorios utilizan. Es un Singleton de *MonoBehaviour* para que persista entre escenas con *DontDestroyOnLoad*.

SaveSlotRepository y *PlayerRepository* son clases C# puras (sin herencia de *MonoBehaviour*) que implementan las operaciones CRUD sobre sus respectivas tablas. Reciben el *DatabaseManager* por constructor, lo que permite cambiar la implementación de la base de datos sin tocar la lógica de negocio.

SaveGameManager es el punto de entrada público que coordina los repositorios. El resto del juego solo habla con *SaveGameManager*; nunca accede directamente a los repositorios ni a *DatabaseManager*.

- **Capa de lógica de juego (Controlador)**

Esta capa contiene los componentes de *MonoBehaviour* que implementan el comportamiento del juego:

GameManager es el Singleton central que da acceso global a *ItemManager* y *TileManager*. Actúa como localizador de servicios para los sistemas que necesitan ser accesibles desde cualquier punto de la escena.

Player gestiona el estado del jugador (su inventario) y la lógica de interacción con el entorno: detectar tiles interactivables en la dirección de movimiento y tirar ítems al mundo con un efecto físico. Delega el movimiento en el componente *Movement* y accede al mundo a través de *GameManager*.

Movement es un componente de un solo propósito: leer la entrada del jugador, mover el *Rigidbody2D* y actualizar el *Animator* con la dirección. Al estar separado de *Player*, puede reutilizarse o modificarse sin afectar la lógica de inventario.

Collectable detecta la colisión con el jugador mediante *OnTriggerEnter2D* y añade el ítem al inventario. Al finalizar, notifica a la UI para que se refresque y destruye el objeto de la escena. Este componente depende de *Item* mediante *[RequireComponent]*, lo que garantiza que nunca existe un *Collectable* sin su correspondiente *Item*.

ItemManager mantiene un diccionario de prefabs de ítems indexados por nombre, lo que permite recuperar el prefab correcto al soltar un ítem del inventario en el mundo.

TileManager encapsula toda la lógica de interacción con el **Tilemap**: determinar si una celda es interactuable, marcarla como interactuada y convertir entre coordenadas de mundo y coordenadas de celda.

- **Capa de presentación (Vista)**

La capa de vista es responsable exclusivamente de mostrar el estado del modelo en pantalla, sin contener lógica de juego.

Inventory_UI controla la visibilidad del panel de inventario y coordina la actualización de los **Slot_UI** que lo componen. Escucha la tecla Tab para abrir y cerrar el inventario, y su método **Refresh()** recorre los slots del inventario del jugador para sincronizarlos con la representación visual.

Slot_UI es el componente más pequeño de la vista: recibe un **Inventory.Slot** y actualiza la imagen del icono y el texto de cantidad. Cuando el slot está vacío, pone el alfa del color a cero para evitar que Unity muestre el cuadrado blanco por defecto.

Diagrama de capas

text

La regla de dependencia fluye siempre hacia abajo: la vista conoce la lógica, la lógica conoce la persistencia y la persistencia conoce los datos. Ninguna capa inferior conoce a las capas superiores, lo que hace que el sistema sea extensible y fácil de mantener.

7. Referencias

Protección de datos y privacidad

- [Reglamento \(UE\) 2016/679 — Reglamento General de Protección de Datos \(RGPD\)](#)
- [Ley Orgánica 3/2018, de Protección de Datos Personales y garantía de los derechos digitales \(LOPDGDD\)](#)
- [Agencia Española de Protección de Datos — Guía para el cumplimiento del RGPD](#)

Propiedad intelectual

- [Real Decreto Legislativo 1/1996 — Ley de Propiedad Intelectual](#)

Clasificación por edades

- [Sistema de clasificación PEGI — Pan European Game Information](#)

Licencias de software

- [Apache License, Version 2.0 — texto oficial](#)
- [Open Source Initiative — definición de licencia de código abierto](#)

Tecnologías utilizadas

- [Unity — documentación oficial](#)

- [SQLite — documentación oficial](#)
- [System.Data.SQLite — documentación del paquete](#)
- [Patrón Singleton en Unity — Game Programming Patterns](#)